

**Tejo Cósmico: Desarrollo de un Videojuego en C++ y Qt 6 como
Aplicación de los Principios de la Programación Orientada a Objetos**

Oscar Trujillo

Facultad de Ingeniería, Universidad de Antioquia

Programación Orientada a Objetos

Profesor: Anibal Guerra

5 de junio de 2026

Resumen

Este documento detalla el proceso de creación de Tejo Cósmico, un videojuego para PC desarrollado en C++ con el framework Qt 6 como proyecto final para el curso de Programación Orientada a Objetos. Esta propuesta es una reinterpretación espacial del tejo el deporte tradicional colombiano en la que el jugador debe enfrentarse a invasores alienígenas durante dos niveles.

El juego pone en práctica los pilares de la POO (herencia, polimorfismo, encapsulamiento y composición), junto con la gestión manual de memoria y el modelado de tres tipos de físicas: parabólica, oscilatoria senoidal y perturbación lateral por viento solar. Además, incluye agentes inteligentes capaces de percibir, razonar, actuar y aprender. El resultado es el desarrollo de unas dos mil líneas de código repartidas en nueve clases, con dieciséis recursos multimedia y un ejecutable completamente funcional para Windows.

Palabras clave: programación orientada a objetos, C++, Qt 6, videojuego, herencia, polimorfismo, agente inteligente, física computacional.

Introducción

Desarrollar un videojuego es, sin duda, uno de los ejercicios más completos para aprender Programación Orientada a Objetos. Te obliga a modelar personajes y elementos con comportamientos únicos, a gestionar interacciones en tiempo real y a diseñar un código limpio que se pueda expandir a futuro. Por eso, el reto del proyecto final en el curso de POO de la Universidad de Antioquia fue precisamente ese: construir un juego desde cero donde demostráramos todo lo aprendido en el semestre.

En este informe detallo el proceso completo de creación de Tejo Cósmico, un videojuego que mezcla el tejo nuestro deporte tradicional y patrimonio cultural con una narrativa de ciencia ficción. El protagonista es “El Mocho”, un campeón boyacense que termina viajando por el sistema solar para enfrentarse a una invasión alienígena. Elegí este concepto con dos objetivos claros: primero, rescatar y darle valor a un elemento cultural propio de nuestro país; y segundo, demostrar que la teoría abstracta de la programación se puede materializar en un producto real, interactivo y motivador para uno como desarrollador.

En la parte técnica, implementé el proyecto en C++ utilizando el framework Qt 6, compilado con MinGW en Windows. Me decidí por Qt debido a su solidez para el desarrollo de aplicaciones de escritorio y por las ventajas de su sistema de gráficos vectoriales QGraphicsScene y QGraphicsView. También, su mecanismo de señales y ranuras (signals and slots) fue esencial para resolver el bucle principal del juego de manera nativa, sin tener que recurrir a librerías externas.

Estructuré este documento en nueve secciones para facilitar la lectura:

La sección segunda presenta el planteamiento del problema y los objetivos, tanto general como específicos. La tercera describe los detalles de los requerimientos funcionales y no funcionales del sistema. La cuarta detalla el diseño y la arquitectura, incluyendo el diagrama de clases y las

decisiones estructurales adoptadas. La quinta sección expone el desarrollo y la implementación, y analizando en los aspectos técnicos más relevantes. La sexta proporciona una guía de instalación y uso. La séptima discute los resultados obtenidos y las pruebas realizadas. La octava expone las conclusiones del trabajo. Por último, la novena sección es la lista de referencias bibliográficas y de recursos consultados.

Planteamiento del Problema y Objetivos

Planteamiento del Problema

El reto principal en el curso de Programación Orientada a Objetos consiste en diseñar y desarrollar un software complejo que ponga a prueba nuestra capacidad para aplicar los pilares del modelo estudiado. Para cumplir con este proyecto integrador, la propuesta fue crear un videojuego desde cero que cumpliera con una serie de requisitos técnicos específicos:

Estructura y POO: Diseñar una jerarquía de clases con herencia propia (más allá de la que ya ofrece el framework gráfico) y aplicar mecanismos de polimorfismo. Física del juego: Programar tres comportamientos físicos completamente distintos para los objetos en pantalla.

Diseño de niveles: Desarrollar dos niveles jugables con mecánicas y dinámicas diferenciadas.

Inteligencia Artificial: Implementar un agente inteligente que responda a las cuatro capacidades clásicas: percibir, razonar, actuar y aprender. El sistema no podía ser solo funcional a nivel de código; debía ofrecer una experiencia de usuario completa. Esto implicó diseñar una interfaz gráfica intuitiva, integrar efectos visuales y sonoros, definir condiciones claras de victoria o derrota, y mantener un código limpio, documentado y versionado correctamente con Git. El verdadero desafío estuvo en articular toda esta teoría abstracta (herencia, composición, gestión manual de memoria y agentes inteligentes) con las herramientas prácticas de desarrollo para lograr un producto final sólido, estable y demostrable.

Objetivo General

El objetivo de este proyecto es crear un videojuego de escritorio desde cero usando C++ y el framework Qt 6. Más allá de hacer un juego divertido, el reto está en aplicar a fondo la programación orientada a objetos (usando herramientas clave como herencia, polimorfismo, composición y encapsulamiento), mientras me aseguré de gestionar la memoria de forma manual. Para coronar el proyecto, diseñé un agente inteligente capaz de pensar y actuar usando las cuatro capacidades clásicas de la IA

Objetivos Específicos

- Implementar tres tipos diferenciados de física para los objetos móviles del juego: trayectoria parabólica para los proyectiles, movimiento oscilatorio senoidal para los obstáculos espaciales y perturbación lateral por viento solar como fuerza ambiental.
- Desarrollar dos niveles con mecánicas y perspectivas visuales distintas, garantizando la transición fluida entre ellos y la persistencia del sistema de puntuación a lo largo de la partida.
- Diseñar un agente inteligente, denominado Comandante Alien, que integre las capacidades de percibir el entorno, razonar sobre la información recopilada, actuar en consecuencia y aprender del comportamiento del jugador para adaptar su estrategia.
- Producir un ejecutable funcional para Windows, acompañado de su código fuente versionado en un repositorio remoto, junto con la documentación técnica que permita su revisión y, eventualmente, su mantenimiento o ampliación.

Especificación de Requerimientos

Requerimientos Funcionales

Para cumplir con los objetivos del proyecto y asegurar una gran experiencia de juego, el sistema incluirá las siguientes funciones clave:

- Cuando el jugador hace clic, dispara un tejo justo en la dirección del puntero del mouse. Este proyectil sigue una trayectoria curva, como un lanzamiento real, porque le afecta una gravedad simulada y la fuerza del viento solar que lo desvía de su camino
- El sistema tiene dos niveles diferenciados: el primero corresponde a una vista lateral con objetivos estáticos (mechas) y obstáculos oscilantes (rocas espaciales); el segundo es una vista cenital donde el personaje “el mocho” enfrenta enemigos en movimiento.
- El sistema debe detectar y procesar el choque entre proyectiles y objetivos, sumando puntos al jugador y eliminando los elementos involucrados de la escena.
- La pantalla incluirá una interfaz de juego (HUD) visible en todo momento. En este panel, el jugador podrá consultar de forma rápida sus puntos acumulados, cuántos tejos le quedan para lanzar, el tiempo que le resta para terminar el nivel y la intensidad actual del viento solar
- Incluye pantallas automáticas de victoria y derrota al finalizar el nivel, permitiendo al jugador reiniciar todo el juego tomando control desde una tecla.
- Para lograr una experiencia más atrapante, el juego contará con música de fondo continua y efectos de sonido para los momentos clave, como la explosión de las mechas, la destrucción de los enemigos y las pantallas de victoria o derrota.

- En el segundo nivel, entrará en juego un agente inteligente llamado **Comandante Alien**, cuyo comportamiento se adaptará en tiempo real a la estrategia y los movimientos del jugador.

Requerimientos No Funcionales

- **Rendimiento fluido:** El videojuego mantendrá una tasa estable de **60 FPS** en computadoras de gama media, garantizando una experiencia de juego suave y sin caídas de rendimiento.
- **Control de versiones:** El desarrollo se registrará en un repositorio remoto de **Git**. El historial de cambios reflejará de forma clara y ordenada la evolución y el progreso real del proyecto.
- **Estándares de código:** Para asegurar un código limpio y legible, se seguirán reglas fijas de nomenclatura: las clases iniciarán con mayúscula, los atributos privados llevarán el prefijo `m_`, y tanto las variables como los comentarios se escribirán en español (respetando únicamente los nombres nativos de Qt).
- **Compatibilidad de texto:** Todos los archivos de código se guardarán con codificación **UTF-8 (sin BOM)**. Así se asegura que caracteres como las tildes y la "ñ" se visualicen correctamente en cualquier pantalla.
- **Distribución independiente:** El archivo ejecutable final se entregará de forma autónoma. Esto significa que incluirá todas las librerías de Qt necesarias para que el usuario pueda abrir el juego directamente, sin tener que instalar nada más en su computadora.

Restricciones

- Tecnologías base: El desarrollo se realizará obligatoriamente en C++, utilizando Qt como el framework principal para la interfaz y los gráficos.
- Control de memoria: Para demostrar un dominio completo sobre el ciclo de vida de los objetos, la gestión de memoria se hará de forma manual utilizando new y delete, dejando de lado los punteros inteligentes estándar.
- Plazo de entrega: El proyecto final se presentará para el 5 de junio de 2026.

Diseño y Arquitectura del Sistema

Arquitectura General

El corazón del sistema es la clase **Juego**, la cual hereda de QMainWindow y funciona como el centro de control que organiza todos los componentes del videojuego. Esta clase central se encarga de administrar la escena gráfica (QGraphicsScene), la vista que el usuario ve en pantalla (QGraphicsView) y las listas dinámicas que contienen a los objetos activos: tejos, mechas, rocas y enemigos. Además, coordina tres temporizadores independientes: uno para mantener el bucle de físicas a unos **60 FPS** estables, otro para la cuenta atrás del nivel y un último encargado de actualizar de forma periódica la fuerza del viento solar

Todos los elementos visuales del juego heredan de QGraphicsPixmapItem, lo que permitió integrar directamente en la escena gráfica de Qt. Gracias a esto, aproveche al máximo las herramientas nativas del framework para el renderizado automático y la detección de choques. Tome esta decisión de diseño para priorizar la reutilización de código ya existente en lugar de construir una jerarquía abstracta desde cero, manteniéndonos alineados con la práctica de evitar abstracciones complejas antes de tiempo.

Jerarquía de Clases

El proyecto está construido sobre seis clases propias que se integran con las herramientas nativas de Qt. La jerarquía y el orden de los componentes se organizan de la siguiente manera:

- Control central: La clase Juego hereda de QMainWindow y funciona como la ventana principal y el cerebro controlador de toda la aplicación.

- Elementos en pantalla: Las clases Tejo, Mecha, Roca y Enemigo heredan directamente de QGraphicsPixmapItem, dándole vida a todos los objetos visuales que interactúan en la escena gráfica.
- El Jefe Final: La clase ComandanteAlien hereda de Enemigo. Esta es la única relación de herencia propia que desarrollamos en el proyecto, diseñada para cumplir con los requerimientos específicos del curso. Su ventaja principal es que recicla la base común de cualquier enemigo (gráficos, puntos de vida y animaciones de destrucción) y la expande agregándole la lógica de un agente inteligente.

Diagrama de Clases – Tejo Cósmico

Proyecto Final – Programación Orientada a Objetos – Universidad de Antioquia – 2026-1

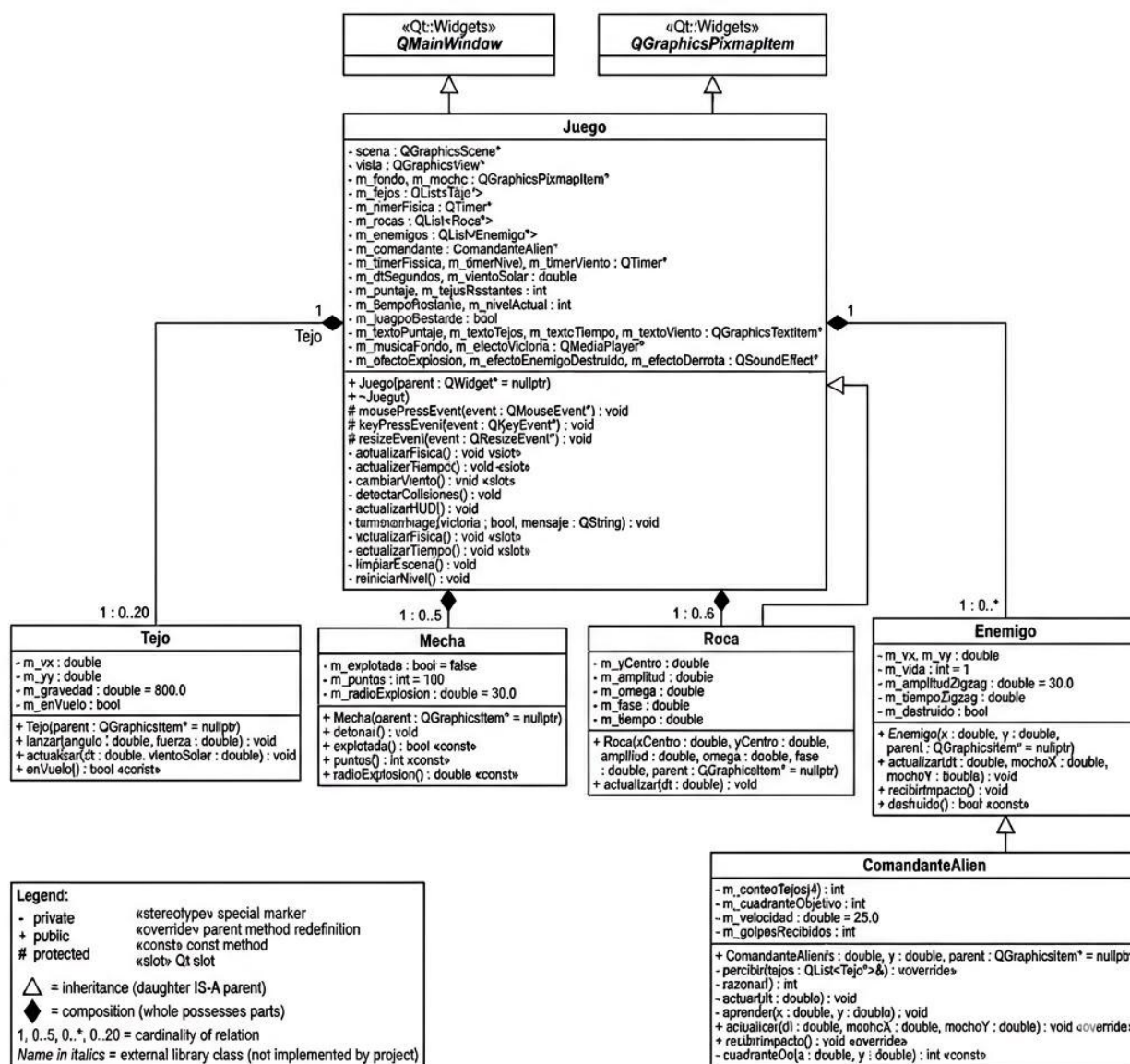


Figura 1. Diagrama de clases UML del sistema Tejo Cósmico.

Decisiones de Diseño

Durante el diseño de este videojuego, tome decisiones claves que son importante dejar documentadas. Cada una representa un balance consciente entre mantener el código simple, la claridad del sistema y cumplir con los objetivos pedagógicos del curso.

Composición Sobre Herencia

Para asegurar la gestión de memoria segura, la clase **Juego** se relaciona con sus componentes mediante una composición fuerte (usando listas de punteros). Así, el ciclo de vida de los objetos está directamente ligado al de la clase principal. Esta estructura previene por completo las fugas de memoria (*memory leaks*), asegurando que el destructor de **Juego** libere automáticamente todos los recursos acumulados. La cantidad de elementos en pantalla se limitó estrictamente por las reglas del videojuego: un máximo de 20 tejos simultáneos, 5 mechas por nivel, 6 rocas y un número variable de enemigos.

Despacho Explícito en Lugar de Métodos Virtuales

Una decisión de diseño importante involucró al método actualizar de la clase Enemigo, el cual decidí no declarar como virtual. Para lograr que el **comandante Alien** hiciera su comportamiento único (siendo más complejo que el de un enemigo común), se implementó un despacho explícito mediante una comprobación de tipo dentro de la clase Juego. Aunque esta solución se aleja del polimorfismo dinámico tradicional, se permitió mantener a la clase base enemigo completamente independiente de su versión especializada. Con esta solución evité que la futura incorporación de nuevos tipos de enemigos obligara a modificar o alterar la clase base

Sprites Generados Programáticamente

Para los elementos dinámicos del juego (como los tejos, las mechas, las rocas y los enemigos, incluido el comandante), decidí no cargar imágenes externas. En su lugar, los genere directamente en tiempo de ejecución utilizando la herramienta QPainter de Qt. Esta elección de

diseño le da al proyecto una gran independencia frente a archivos externos y, al mismo tiempo, hace que el ejecutable final sea mucho más ligero y fácil de distribuir. Los únicos archivos de imagen que el juego necesita cargar de forma externa son los de los elementos estáticos: el fondo del escenario y el personaje principal.

Desarrollo e Implementación

Tecnologías y Entorno de Desarrollo

El desarrollo se llevó a cabo utilizando las siguientes tecnologías:

- Lenguaje de programación: C++ estándar 17.
- Framework gráfico: Qt 6.10.2 (módulos Widgets, Multimedia y MultimediaWidgets).
- Compilador: MinGW de 64 bits, distribuido con la instalación oficial de Qt.
- Sistema de construcción: CMake versión 3.16 o superior.
- Entorno de desarrollo integrado: Qt Creator.
- Sistema de control de versiones: Git, con repositorio remoto en GitHub.
- Sistema operativo de desarrollo y compilación: Windows 11.

Las Tres Físicas Implementadas

Uno de los requisitos centrales del proyecto consistió en implementar tres tipos diferenciados de física. A continuación se describe cada una.

Física Parabólica del Tejo

El movimiento del tejo se calcula usando las fórmulas tradicionales del tiro parabólico. En cada cuadro del juego, el sistema actualiza su velocidad sumándole una gravedad simulada de (ochocientos píxeles por segundo al cuadrado) y el intervalo de tiempo y ajusta la velocidad horizontal según la fuerza del viento solar del momento. Debido a que el sistema de coordenadas de Qt tiene el eje vertical invertido (el origen está arriba), la velocidad vertical inicial se configura con un valor negativo para lograr que el tejo suba correctamente en pantalla.

Física Oscilatoria Senoidal de las Rocas

Programé las rocas para que se desplacen de arriba a abajo siguiendo un movimiento armónico continuo basado en la función seno. Para lograr que el escenario se vea mucho más dinámico, le asigné a cada roca sus propios valores de velocidad, altura de oscilación y punto de partida; esto evita que se muevan al mismo tiempo y crea un patrón de obstáculos impredecible y atractivo en la pantalla.

Perturbación Lateral por Viento Solar

Implementé el viento solar como una fuerza ambiental que afecta directamente la trayectoria de todos los tejos que están en el aire. Programé el sistema para que, cada cuatro segundos, calcule de forma aleatoria una aceleración horizontal de entre menos ciento cincuenta y más ciento cincuenta píxeles por segundo al cuadrado. Si este valor resulta inferior a veinte píxeles por segundo al cuadrado, el juego considera que el viento está en calma. Con esta variación constante, busco obligar al jugador a ajustar y compensar el ángulo de su lanzamiento según el clima del momento, añadiendo una capa táctica muy interesante a cada partida

El Agente Inteligente: Comandante Alien

Para el segundo nivel del juego programé un agente inteligente llamado Comandante Alien como una clase derivada de Enemigo. Para estructurar su comportamiento me basé en el marco teórico de Russell y Norvig (2021), aplicando las cuatro capacidades clásicas de los agentes racionales, que implementé de la siguiente manera:

Percibir

El agente recibe las coordenadas exactas de cada lanzamiento del jugador. Diseñé el juego para que, cada vez que se detecta un clic del mouse, la clase Juego llame al método aprender y le envíe esta información al Comandante.

Razonar

Dividí la pantalla del juego en cuatro cuadrantes y le asigné al agente un contador interno para registrar cuántos tejos caen en cada zona. A través del método razonar, el Comandante analiza estos datos, identifica cuál cuadrante ha recibido menos ataques y lo marca como su objetivo seguro para moverse.

Actuar

Programé al agente para que se mueva hacia el centro del cuadrante objetivo a una velocidad baja de 25 px/s. Esta lentitud en comparación con los enemigos comunes (80 px/s) representa, de forma metafórica, el "costo de pensar" o procesar la información frente al instinto puro.

Aprender

Cada vez que lanzas un tejo, el sistema suma un punto al contador del cuadrante afectado. Así, el Comandante acumula experiencia en tiempo real y adapta su comportamiento para huir de tus zonas preferidas de tiro y buscar las menos vigiladas

Gestión Manual de Memoria

Decidí gestionar la memoria de los objetos dinámicos de forma explícita con new y delete, dejando de lado los punteros inteligentes. Libero cada objeto manualmente cuando choca, sale

del mapa o cuando limpio la pantalla. Para esto último, diseñé el método `limpiarEscena`, que recorre las listas del juego (tejos, enemigos, rocas, mechas y HUD), borra cada elemento y vacía las listas antes de pasar al siguiente nivel para demostrar un control total del ciclo de vida del software.

Guía de Instalación y Uso

Requisitos del Sistema

- Sistema operativo: Windows 10 o superior (versión de 64 bits).
- Memoria RAM: 2 GB como mínimo.
- Espacio en disco: 150 MB para la versión empaquetada con todas sus dependencias.
- Tarjeta de sonido compatible para reproducción de música y efectos.

Instalación

Para instalar el juego en su versión compilada, se descomprime el archivo ZIP de distribución en cualquier carpeta del sistema. Posteriormente, se ejecuta el archivo TejoCosmico.exe. No se requieren permisos de administrador ni la instalación previa de Qt en la máquina destino, ya que el paquete incluye todas las bibliotecas dinámicas necesarias junto con el ejecutable.

Para quien desee compilar el proyecto desde el código fuente, los pasos son los siguientes: clonar el repositorio de GitHub, abrir el archivo CMakeLists.txt con Qt Creator, seleccionar un kit de compilación compatible con Qt 6.10 o superior, configurar el proyecto y ejecutar la construcción mediante la combinación Ctrl+R. Las dependencias se resuelven automáticamente mediante CMake y los recursos gráficos y sonoros se empaquetan en el binario a través del sistema de recursos integrado de Qt.

Controles del Juego

- Clic izquierdo del mouse: lanzar un tejo en la dirección del cursor.
- Tecla R: reiniciar el nivel actual.
-

Mecánicas del Juego

Nivel 1: Cancha Lunar

(Vista Lateral): Diseñé este nivel con el protagonista ubicado a la izquierda de la pantalla y cinco mechas como objetivos al lado derecho. Para añadir dificultad, coloqué tres rocas espaciales que se mueven de arriba a abajo y destruyen los tejos sin dar puntos. Le di al jugador 10 tejos y 90 segundos para romper las 5 mechas (cada una da 100 puntos) antes de que se le acaben los recursos o el tiempo.

Nivel 2: Anillos de Saturno

En esta etapa cambié la perspectiva y posicioné al protagonista en todo el centro. Configuré enemigos que aparecen desde los cuatro bordes moviéndose en zig-zag hacia él, acompañados por el Comandante Alien, que se desplaza de forma inteligente esquivando las zonas donde más suelo atacar. Cada enemigo común da 200 puntos, y el Comandante (que aguanta tres golpes) da 500 puntos. Para ganar, debo eliminar a todos los invasores; si uno solo llega a tocarme, pierdo la partida.

Resultados y Discusión

Funcionalidades Logradas

Al cerrar la etapa de desarrollo, logré un producto completamente funcional que cumple con todos los requisitos solicitados. El juego corre de forma estable a unos 60 FPS en computadores de gama media, y comprobé que no presenta fugas de memoria ni errores durante sesiones largas de juego. El ejecutable final, empaquetado con todas sus dependencias, pesa solo 109 MB en un archivo comprimido.

En términos métricos, escribí cerca de 2,000 líneas de código organizadas en 9 clases (6 creadas por mí y 3 heredadas de Qt). Además, integré 16 recursos multimedia (8 imágenes PNG y 5 archivos de audio entre WAV y MP3) y registré un total de 17 commits en mi repositorio de Git a lo largo del semestre.

Pruebas Realizadas

Durante el desarrollo, realicé pruebas funcionales manuales a medida que programaba cada componente. De esta forma, verifiqué que la física parabólica del tejo, la oscilación de las rocas y la detección de colisiones funcionaran sin problemas. También validé la actualización en tiempo real del panel de información, las transiciones de nivel y el audio. Un punto clave fue evaluar el aprendizaje del comandante Alien; pude observar cómo el agente inteligente analizaba mis tiros y evitaba progresivamente las zonas donde yo lanzaba con más frecuencia.

Limitaciones Encontradas

Por restricciones de tiempo, no pude implementar las mecánicas especiales del protagonista que habíamos diseñado en papel (como Guarapero y Ojo de Águila). Sin embargo, gracias a la arquitectura que se contruyó, se pudo integrar en el futuro heredando de la clase principal sin alterar el código existente. El agente inteligente cumple con las cuatro capacidades solicitadas para el laboratorio, pero su sistema de aprendizaje actual es simple (usa un histograma de frecuencias por cuadrante). Me gustaría potenciarlo más adelante utilizando técnicas avanzadas como redes neuronales básicas o algoritmos de Q-learning.

Trabajo Futuro

Para continuar con la evolución del sistema, he propuesto las siguientes metas:

- Evolución del personaje: Activar las habilidades especiales del protagonista.
- Expansión del juego: Crear un tercer nivel y un sistema que guarde los puntajes máximos de los jugadores.
- IA Avanzada: Potenciar al agente inteligente usando aprendizaje por refuerzo para que sea un rival más desafiante.
- Multiplataforma: Exportar el juego a Linux y macOS para aprovechar al máximo las capacidades nativas de Qt.

Conclusiones

El desarrollo de *Tejo Cósmico* me permitió consolidar los pilares de la programación orientada a objetos (POO) en un entorno real. Integrar conceptos como herencia, encapsulamiento, gestión manual de memoria y un agente inteligente en un solo software demostró el valor práctico de estas herramientas.

Usar Qt me enseñó cómo un framework acelera el trabajo al delegar tareas complejas como los gráficos y el audio. Sin embargo, también me recordó la importancia de entender sus reglas internas para adaptar el código correctamente.

Este proyecto nos recuerda que programar también es diseñar. Elegir una buena estructura de clases, normas de orden y un control de versiones desde el primer día fue lo que permitió mantener un ritmo de desarrollo constante y sin tropiezos. Finalmente, reinterpretar un deporte tradicional colombiano bajo una temática de ciencia ficción demostró que el código también es un espacio de expresión cultural. Llevar a un protagonista boyacense a viajar por el espacio simboliza la esencia de este laboratorio: tomar lo nuestro y proyectarlo hacia nuevos horizontes tecnológicos.

Referencias

- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.
- Meyers, S. (2014). Effective Modern C++: 42 Specific Ways to Improve Your Use of C++11 and C++14. O'Reilly Media.
- Russell, S. J., & Norvig, P. (2021). Artificial Intelligence: A Modern Approach (4^a ed.). Pearson.
- Stroustrup, B. (2013). The C++ Programming Language (4^a ed.). Addison-Wesley.
- The Qt Company. (2024). Qt 6 Documentation. <https://doc.qt.io/qt-6/>
- Kitware. (2024). CMake Documentation. <https://cmake.org/documentation/>
- Chacon, S., & Straub, B. (2014). Pro Git (2^a ed.). Apress. <https://git-scm.com/book>
- Instituto Colombiano de Cultura. (2010). El tejo: deporte autóctono de Colombia. Ministerio de Cultura.